

State and Action Formulation — SIH26055 Scan Scheduler

Version 1 · 2026-09-06 · Owner: Aamir. Audience: the RL lane, as the input to the reward / PPO work.

AMENDED 2026-09-09 — **three things below are now out of date; see D52, D53 and D54.** (1) ~~The registered reward candidates are `hit_z`, `hit_y`, `reward_balance`.~~ **CORRECTED 2026-09-10:** the live registry is `reward_balance`, `greedy`, `explore`, `weighted`. `hit_z` and `hit_y` — this document's default and its worked comparison reward throughout — were **retired entirely** after both failed D62's screen (point 7, below); every `--reward hit_z` or `--reward hit_y` command anywhere in this document now raises. The key `first_intercept` also no longer exists — candidate 3 was renamed and rewritten (D50, D53). (2) `reward_balance`'s camping cost is charged against `airtime share` ($-3.0 * \text{visit_density}[\text{action}] * \text{n_slots}$), not a consecutive-repeat streak — the streak version was defeated for free by alternating between two bands, and measurably ranked a 2-band ping-pong above round-robin (D53). (3) Inference **samples** the policy; it no longer takes the `argmax`. `RLScheduler` and `RecurrentRLScheduler` take `deterministic`, defaulting to `False`, so every code snippet below showing `deterministic=True` is stale except for rung 7 (a DQN's greedy action is its policy). (4) The observation is **183 wide**, and its box is no longer `[0, 1]` (D55, D67): `camp_time` was dropped as measurably inert, `visit_density` now reads in fair shares (ceiling 36.0) and staleness in reference sweeps (ceiling 13.95). Every checkpoint that predates this is unloadable. (5) `reward_balance` separates catastrophe from competence but not competence from excellence over round-robin ~~-- measured, it scores rung 5 and round-robin within $+2.3 \pm 11.7$ of each other over 8 seeds, so round-robin is roughly the ceiling it can teach (D56, open).~~ **WITHDRAWN 2026-09-10 -- that measurement was wrong (D56).** It scored a hand-written step `% N_BANDS` sweep and called it round-robin; round-robin is `EQUAL_AIRTIME_CYCLE` (D43). Against the real rung, `reward_balance` separates rung 5 from round-robin by **$+59.0 \pm 19.4$ on 8/8 seeds** -- three times the seed noise, not three percent of it.

(6) The registered set grew to **six** under D57 (`hit_z`, `hit_y`, `reward_balance`, `greedy`, `explore`, `weighted`), then back down to **four** when `hit_z/hit_y` were retired (point 7). D29's three-candidate cap is formally lifted for the designed axis D57 added; `greedy/explore` are the exploit/explore corners and are *expected* to fail on their own, `weighted` ($\alpha=0.3$) is the knob between them.

(7) Every candidate is now screened before anything trains on it (D62): rungs 2, 4, 5, 6a scored 8 seeds each, PASS only if rung 5 clearly separates from round-robin and the camper sits clearly below both. **`hit_z` and `hit_y` -- this document's own default and worked reward -- both FAILED**, ranking the camper above every sweeping policy, and were **removed from REWARDS as a consequence** (point 1). `reward_balance` is the only survivor. Run `python -m rfenv.reward_gate` before training on anything else here. D47's selection rule is gated behind this screen and has still never been run.

(8) Training no longer samples the full 47-config pool (D60). The train/eval leak this caused -- the RL policy could see the same emitters it was scored against, which the baselines never could -- is closed: `EmitterPool.from_train()` (all 47) is now the *evaluation* pool only; `split.training_pool()` (35 configs) is what `make_train_env` actually trains on, with **zero** emitters shared with the 12-config validation half. **No RL result in this document predates the split and none of them is clean.**

(9) Checkpoint selection is pre-registered (D61): highest $P(\text{dominates rung 5}) - P(\text{dominated by rung 5})$ on the validation half, fixed in `rfenv/selection.py` before the run that uses it -- not picked by eye from a compare table.

(10) `measured_dbm` is ratified (D59) and stays in the observation.

AMENDED 2026-09-14 — (11) The 183-wide observation described in this document is now called "**v1**", and it is still the default (`ScanEnv(obs_version="v1")`, implicit) — every checkpoint this document describes stays exactly as loadable as it was. A second, opt-in "**v2**" layout also exists now (D30 resolved as D75): 326 wide, adding `PulseWidth` and `AoA` (`sin θ`, `cos θ`, gated on a declared hit) plus an upgrade of `measured_dbm` from one global last-dwell scalar to a per-band block. §4's 109 → 146 → 183 progression does **not** gain a fourth entry in the same line — 326 is a sibling of 183, not its successor. No checkpoint has trained on "v2" long enough to report a result as this is written.

AMENDED 2026-09-14 — (12) "v2" widened again the same day, on a direct request: **362 wide**, not 326. `pulse_count` (D76) is a sixth "v2"-only block, γ -gated the same way `PulseWidth`/`AoA` are — the illumination count `C` (`truth.py`) at the loudest slot of the band's last declared hit, `log1p`-normalised against the same reference `reward_balance_improved` already uses for `C` on the reward side, clipped to `[0, 1]`. This reopens D29/D34's own exclusion of pulse count from the observation; the reasoning for reopening it, and the one gap gating narrows but does not close (`C` itself carries no gamma gate, so a hit's count can still include sub-threshold co-located emitters), is recorded in `DECISIONS.md` D76. `known_observation_widths()` now returns `{183, 362}`, not `{183, 326}`. The three "v2" checkpoint snapshots D75's own retrain had already produced at 326 wide are now permanently unloadable — the same cost every past observation change has carried, this time paid by "v2" rather than "v1".

AMENDED 2026-09-18 — (13) A third, opt-in layout, "**v2p**", now exists: "v2" (362 wide) plus one more 36-wide block, `band_priority` — **398 wide total**. Unlike every block above, it is not derived from anything the receiver measures: it is a per-episode input, `1.0` = ordinary band, `3.0` = elevated, either sampled fresh each episode (3–6 of the 36 bands elevated) or held at all- `1.0` for a control arm (`priority_uniform=True`). Paired with it, `ScanEnv.step()` adds a reward term directly (`reward += priority_coef * band_priority[dwell.band] * len(newly)`) on top of whichever `REWARDS` candidate is selected — additive, gated on **discovering something new** rather than on occupying the band, which is what keeps it clear of D53's per-slot camping exploit. This is **not** a new `REWARDS` entry and does not touch `reward_gate.py`'s D62 screen. Two RecurrentPPO checkpoints (ladder rungs 22a treatment / 22b control) were trained and compared 2026-09-18 to test whether a policy actually conditions on `band_priority` rather than merely benefiting from the reward-scale increase it adds uniformly. **The result runs the wrong way**: the control (no real signal) beat the treatment on every headline column, 61.4% vs 44.4% beats-recency-both. Camping was ruled out directly (~7% more airtime on elevated bands, inconsistent across episodes); treatment was measurably more diffuse than control across the whole spectrum (entropy 3.255 vs 3.051, 24.95 vs 18.95 of 36 bands touched); and a **permutation ablation settled it 2026-09-19**: the treatment checkpoint's airtime correlates with true priority equally poorly whether fed the real vector or one shuffled across bands (+0.018 vs +0.019), so **the agent never learned to use band_priority** — the underperformance is a training-difficulty story (control's input is constant, an easier problem), not a misused-signal one. Single seed, not read as settled beyond this configuration. **DECISIONS.md D78, MEASURED** — unlike (11)/(12) above, written up as **SETTLED** the day they were built, this write-up was deliberately held until both the comparison and the ablation existed. Not adopted, not promoted, code not removed. Full mechanism: `OBSERVATION_SPACE.md` §2.4; numbers: `MODEL_COMPARISON.md` Width 398.

AMENDED 2026-09-19 — (14) Same-day follow-up to (13): a new decaying per-slot occupancy term (`priority_reward_bonus`, `env.py`, anchored to `visit_density` rather than a resettable streak to stay clear of D53), `priority_coef` 0.5→2.0, the elevated value now configurable (`priority_high`, was hardcoded) raised 3.0→5.0, LSTM hidden size doubled 256→512, timesteps doubled 400k→800k (rungs 23a/23b). Result: treatment **73.1%** beats-recency-both, the highest figure measured in this project — but a second permutation ablation found the same thing as the first: airtime correlates with true priority identically whether fed real or shuffled values (+0.031 both

ways). **Still never learned**, at roughly 4x the incentive and double the capacity/budget; 23a's lead over its control (45.6%) is read as training-run variance, not the mechanism, since the one thing actually measured — whether the signal is read — stayed "no" both times the question was asked. Verdict unchanged: not adopted, not promoted, code not removed. Same DECISIONS.md D78 entry, extended, not a new decision number.

AMENDED 2026-09-19 — (15) A fourth layout, "v3" (436 wide), and with it two changes to the episode itself. ScanEnv(obs_version="v3") is "v2p" plus prev_action (36, one-hot of the last band, all-zero before the first step), prev_reward (1) and prev_hit (1) — the RL² interface, so a recurrent policy can adapt inside the episode with no gradient step (D79). **36 of those 38 columns duplicate blocks that already existed:** prev_action is bit-identical to current_band after any step and prev_hit is current_hit_streak > 0, so prev_reward is the only new information and any claim about "v3" should say so. prev_reward carries reward_balance_obs, which is reward_balance with Y for Z — the training reward would leak Z into the observation, since the agent already holds the other three terms and could solve for it. **D29 is unchanged:** the reward still reads truth and still scores every arm.

ScanEnv(episode_slots=...) lets one episode run longer than a recording, with the world stitched from independent 30 s draws (D80). constants.py is untouched — D42's freeze holds and every default-length episode is bit-identical, pinned by golden digests taken before the change. Two blocks are now bounded rather than unbounded (clock divides by this episode's length, staleness is clipped to its declared ceiling at both the observation and the reward site), and episode_metrics() censors a missed emitter at its own segment end rather than at the whole mission — a correctness fix at length that evaluates to exactly 600 slots at the default.

rfenv/rl/online.py fine-tunes a checkpoint while it scans, on a continuous grid, with the gradient fed reward_balance_obs rather than step()'s truth-fed return (D81). Per-mission updates are deliberately not offered: 300-600 decisions cannot fill an n_steps=8192 rollout.

AMENDED 2026-09-20 — (16) Amendment (15)'s prev_action block, above, was removed from "v3" the next day, narrowing it 436 → 400 wide **in place** (the same class of width change D49/D55/D67/ D76 made — lstm_v3_seed0/lstm_v3_seed1, both complete 800k-step checkpoints, are now permanently unloadable). Asked directly why the observation needed prev_action when an LSTM's hidden state already carries information forward: it can only carry forward what appeared in its input, and current_band — in every layout since "v1" — already put the last action there at every step, so prev_action was duplicating a channel the network already had, not adding one. prev_reward is unaffected and remains "v3"'s one genuinely new column; prev_hit stays too, lacking the same direct duplicate. Same DECISIONS.md D79 entry, extended.

AMENDED 2026-09-20 — (17) Asked directly whether offline training is still available after D81 (online fine-tuning): yes, completely unaffected, and it stays the default. Everything this document describes — the observation, the action space, every layout including "v3" — is built by the ordinary rfenv.rl.{ppo, recurrent_ppo, dqn}.train() path exactly as before; rfenv/rl/online.py only ever loads an already-trained checkpoint from that path and keeps adapting it.

AMENDED 2026-09-20 — (18) Amendment (15)'s prev_hit block, above, was removed from "v3" too, the same day as (16), narrowing it 400 → 399 wide **in place** — requested directly, ahead of the layout's first training run, to isolate prev_reward's own effect rather than test two additions at once. prev_hit was already on record (amendment (15)) as "pre-existing information" with no companion block making the redundancy as directly provable as prev_action's, but had been kept anyway; asked plainly whether that was a reason to keep it or just a weaker excuse, it was removed. **prev_reward is now the only block "v3" carries beyond "v2p"** — no checkpoint cost this

- **36 frequency bands**, centres 250 MHz $\rightarrow$ 17,750 MHz on 500 MHz spacing (`rfenv/constants.py:20`, read from `metadata/receiver/dwell_centres_mhz`; verified identical across all 47 train scan configs). Half-width is **± 500 MHz**, so **adjacent bands overlap by half** — D3's measurement: 99.985% of 4,393,233 train scan pulses fall within ± 500 MHz of the dwell centre active at their ToA, against 50.92% for a disjoint ± 250 tiling.
- **A 30 s episode divided into 600 slots of 50 ms** (`rfenv/constants.py:43-45`). 30.0 s is Turing's own `collection_time_s`, identical across all 47 train pairs.

- **A binary world status per (band, slot)** — the PS's own framing: *"The status of environment for each frequency band at each time step can be recorded as a transmission or a non-transmission."*
- **No prior knowledge whatsoever.** Every episode starts cold — no threat library, no carry-over, no prior on who transmits where (**D20**). This is the PS's *"absence of prior reliable intelligence"* made literal, and it is what keeps the explore/exploit tension real: with a prior, camping on the busiest band would be defensible.

Looking at one band means **not** looking at the other 35, and an emitter that transmits while the receiver is elsewhere is missed. Against **spatially scanning and frequency agile** emitters, a band that was empty a second ago may be busy now. That is the whole problem.

2. STATE — what the agent knows at each timestep

2.1 The observation space

```
observation_space = spaces.Box(low=low, high=high, dtype=np.float32) # shape (183,)
```

183 = 36 × 5 + 3, and the box is NOT the unit interval (D55, D67). Two blocks declare ceilings above 1.0 so that 1.0 means something inside each — `visit_density` reaches `N_BANDS = 36.0` (a fully camped episode) and `staleness` reaches `N_SLOTS / SWEEP_SLOTS = 13.95` (a band untouched all episode). Everything else is a fraction or a one-hot and **no scaling or normalisation layer is needed anywhere.**

D34 ratified the base **109 = 36 × 3 + 1** (hit rate, visit density, staleness, clock). **D49 extended it** with `current_band` (36-wide one-hot of the band just dwelt on), `camp_time` (consecutive slots on that band / `N_SLOTS`) and `measured_dbm` (the last dwell's mean measured level, clamped and rescaled); **D55 then dropped `camp_time` as measurably inert and rescaled `visit_density` and `staleness`; D67 appended `hit_streak`** (36-wide, consecutive declared hits per band across visits, capped and rescaled) **and `current_hit_streak`** (that scalar for the band just dwelt on) — 109 → 145 → 146 → 147 → 146 → 183, in that order. D34 remains the base; D49 and D67 are the extensions and the reason for the current width. Each extension appended rather than reordered, so every earlier slice offset in `baselines/guard.py` is unchanged and no heuristic rung needed touching — but **every checkpoint trained before an extension is permanently unloadable**, which is the cost D49, D55 and D67 each paid deliberately. Note that **every trained checkpoint breaks at each such change**: SB3 sizes a policy's input layer at construction, so a width change is a retrain, not a reload.

Layout, in order (`rfenv/env.py::_observation`):

Index	Component	Exact definition	Type / range
<code>0:36</code>	per-band hit rate	<code>hits[b] / slots_looked[b]</code> ; <code>0.0</code> if band never looked at	float32, <code>[0, 1]</code>
<code>36:72</code>	per-band visit density	<code>slots_looked[b] / max(t, 1)</code> — fraction of spent airtime given to band <i>b</i> . Sums to exactly 1 across bands once <code>t ≥ 1</code> ; all-zero in the <code>reset()</code> observation	float32, <code>[0, 1]</code>

Index	Component	Exact definition	Type / range
72:108	per-band staleness	$(t - \text{last_slot}[b]) / 43$, i.e. in reference sweeps (D55); $600/43 = 13.95$ if never visited	float32, [0, 13.95]
108	normalised episode time	$t / 600$	float32, [0, 1]

Never-visited bands read staleness 13.95 — maximally stale, the whole episode expressed in sweeps (verified: at `reset()`, all 36 staleness components are exactly 1.0). So an unexplored band looks at least as attractive as one last seen at $t = 0$. That is a deliberate exploration prior baked into the encoding.

2.2 Each state variable against the five criteria

Variable	Why it is relevant	Available / observable in our env?	Agent-controllable?	Type / range	Connection to the PS objectives
Hit rate (36)	What a scheduler needs to estimate detection probability for a band — how likely a look here pays off. It is the exploit signal	Yes , computed from the agent's own dwell outcomes only	Indirectly — the agent moves it by choosing where to look	float32 [0, 1], one per band	PS figure of merit <i>probability of detection</i> . And directly PS: " <i>The model should then be trained based on hits and misses</i> " — this is literally that
Visit density (36)	How the agent's airtime is being spent. Airtime is the only currency in this problem (§3.2), so this is the agent's own budget, made legible	Yes , from own scan history	Yes — it is a direct summary of its own past actions	float32 [0, 1], sums to 1 for $t \geq 1$ (0 at reset)	PS figure of merit Avg <i>intercept rate</i> ; supports high interception rate
Staleness (36)	What makes this restless rather than a plain bandit. A band ignored for 10 s may have become busy without telling you. The payoff distribution moves while you are not looking. It is the explore signal	Yes , from own scan history	Yes — looking at a band resets its staleness to 0	float32 [0, 13.95], in sweeps (D55)	PS: scheduling " <i>against spatially scanning and frequency agile emitters</i> "; drives minimise intercept time

Variable	Why it is relevant	Available / observable in our env?	Agent-controllable?	Type / range	Connection to the PS objectives
Normalised time (1)	Lets the policy behave differently early (explore) and late (exploit). The episode is finite and terminates — there is no state past slot 600	Yes , trivially	No — the clock advances regardless of what it picks	float32 [0, 1]	Both objectives; intercept time is measured against a fixed 30 s horizon

All four are built from the agent's own scan history and nothing else. No prior emitter intelligence enters (**D19**, **D20**), and nothing truth-side leaks into the policy's input path (**D29**).

2.3 The hard constraint the RL lane must not break

This is **D29**, and it is the single most important rule for this lane:

The reward may read truth. The observation may not.

Training is offline and the policy is frozen before deployment, so the reward is a **training-time construct discarded at inference**. It may read the truth grid Z, per-emitter signal levels, `first_e` — anything. The **observation** may not, because the 183-vector is all a fielded receiver would actually have.

This is enforced in code, not by convention. `rfenv/baselines.py:69`:

```
OBSERVABLE_INFO = frozenset({"slot", "time_s", "band", "dwell_slots", "Y"})
```

`baselines.guarded()` strips info down to that set before any policy sees it, so a policy reaching for Z, pulses or `first_intercept` raises a `KeyError` **instead of quietly scoring well**. `baselines.make()` wraps every deployable rung in `guarded()` automatically.

RL lane: wrap your policy in `guarded()` too. A `KeyError` catching an information leak is far better than a suspiciously good score catching it.

2.4 What the state deliberately excludes, and why

Excluded	Why
Pulse count, peak amplitude within the dwell	Available at L2 and richer than the PS's framing, but the PS states the environment status as binary — <i>"a transmission or a non-transmission"</i> (D34)
Anything per-emitter	D19 : any design needing per-emitter attribution from observations inherits a deinterleaving problem . Avoid needing it. Deinterleaving is explicitly not the deliverable (D12)
Truth grid Z, <code>first_e</code> , per-emitter levels	D29 — reward-side and evaluator-side only

Excluded	Why
Any prior threat library	D20 — cold start every episode
AoA and PulseWidth	D30 — still open, and owned by the RL lane. See §4.

3. ACTIONS — what the agent can actually decide

3.1 The action space

```
action_space = spaces.Discrete(36)      # rfenv/env.py:159
```

Criterion	Answer
What it is	"Tune the receiver to band <i>a</i> and dwell there for that band's native dwell time"
Why relevant	It is the <i>entire</i> lever the receiver has. The PS's two search dimensions are frequency and time; choosing a band now is choosing both
Available in our env?	Yes — implemented, and all seven baseline schedulers drive the env through it
Agent-controllable?	Yes. This is the only thing the agent controls (see §3.4)
Type / range	Discrete integer, 0 ... 35, one per frequency band
Legality	All 36 actions are legal at every timestep — no masking layer is needed anywhere (D35)
PS connection	Directly serves both objectives: where you point determines both whether you intercept at all (interception rate) and how soon (intercept time)

3.2 An action's cost is not uniform — this is load-bearing

A band's **dwell length is a property of the band, not a separate choice**. Seven bands — **0, 1, 6, 7, 17, 18, 19** — carry a 100 ms dwell and consume **2 slots**; the other **29** carry 50 ms and consume **1 slot** (`rfenv/constants.py:33-37, 51`, read from `metadata/receiver/dwell_times_s`; re-verified at runtime 2026-09-06). Turing's full sweep of all 36 is **2.15 s**.

Three consequences the RL lane must not get wrong:

- An episode is 600 slots but 300–600 `step()` calls (**D35**)**. Camping on band 0 (wide) takes 300 steps; camping on band 5 (narrow) takes 600. Both cover exactly 30 s. So anything computed *per step* — advantage normalisation, entropy schedules, "average reward per step" — is measuring something different from what the metrics measure.
- Airtime is the only currency**. Retuning is free — D31's measurement puts it under 1 μ s, i.e. under 0.002% of a slot. A wide band genuinely costs twice as much of the episode as a narrow one.

3. **Therefore reward is per slot, and a dwell's reward is the sum of its slots' (D31).** A wide band can earn up to +2. The naive "+1 per dwell" is **wrong** and was explicitly rejected: it would make those seven bands strictly dominated — and per D31 they are the bands that matter most, carrying a recorded mean of 764.6 emitter-frequency placements against 170.4, and the slowest rotators (bands 0 and 1 at median 3.00 rpm).

3.3 An overrunning dwell is clipped, not made illegal

A wide-band action at slot 599 is clipped to one slot rather than forbidden (D35), so the action space never changes shape mid-episode and no policy needs a masking layer for one edge case.

3.4 What is deliberately NOT an action, and why

This is the part most likely to be read as something missing. The agent does **not** choose any of the following, and each has a recorded reason:

Not an action	Why not
Dwell length / bandwidth	Fixed per band by Turing's own receiver configuration, adopted unchanged (D3). Reproducing that configuration exactly is what makes our numbers comparable to their recordings
Detection threshold γ	$\gamma = -111$ dBm is a frozen receiver property , not an agent knob (D15, D21, D42). It is calibrated and swept as an experiment, never learned. No reward and no policy can move P_d or P_{fa}
Antenna steering / look direction	This receiver tunes; it does not steer . The PS frames the search as frequency $\times$ time only, so AoA can never be something a dwell is spent on (D30)
Number of bands / band geometry	Frozen (D42), enforced by <code>tests/test_freeze.py</code> with per-value literals plus a SHA-256 digest. Changing it forces re-validation from gate 1 and a re-run of every baseline (D25)

So: "what controllable parameters does the agent have?" → **exactly one, which band to look at next**. The richness of this problem is not in the action space. It is in *when* and *in what order*, across 300–600 sequential decisions taken with no prior.

4. The one genuinely open state question: D30

D30 (OPEN) — do AoA and PulseWidth enter the observation?

DECISIONS.md marks this "**owned by the RL lane**"; RL_LANE_HANDOFF.md §5 assigns it to role **R3**. It is the only unanswered state-space question, and it is deliberately unanswered.

The case for. To an agent seeing only binary hit/miss, a dwell that finds a **new** emitter and one that re-finds a **known** emitter are indistinguishable. D30's recorded walk found that under a camper policy ~99% of dwells are redundant and *every one of them looks like a hit*. AoA separates them: assigning each pulse to the emitter with the nearest median bearing scored 96.7% (config_2), 99.3% (config_59) and 86.1% (config_921) against true labels.

The case against. That accuracy is a **ceiling**, not an achievable figure — it used true labels and whole-episode medians. It degrades exactly where the problem is hardest (96.7% → 86.1% going from 19 to 99 emitters). And it costs real structure: L1's grid combines cells by max, which is meaningless on per-emitter bearings, and the fixed-width observation vector

would need a variable-length bearing set encoded by binning or online clustering.

The trigger that decides it: a trained agent failing to explore in a way that hit rate, visit density and staleness demonstrably cannot fix. Measurable **once a policy exists**, meaningless before. `PulseWidth` rides on the same decision.

Direct consequence for the reward lane — read this before choosing a reward. One of the three reward candidates is `first_intercept` (+1 per first intercept of an emitter, D29 candidate 3). Novelty is a **truth-side** quantity: the reward can see it, the observation cannot. Choosing that reward therefore trains the agent to value something it **provably cannot perceive at inference** — it would have to approximate novelty with staleness, which measures *time since looked*, not *have I already found everyone here*. `hit_z` and `hit_y` do not have this problem. If the reward lane lands on `first_intercept`, **D30 stops being optional** — AoA is the observable that would make it actionable rather than merely scorable.

Why it is safe to leave open: the observation vector is **explicitly not on the freeze list** and the four validation gates never read it. Adding AoA later costs a retrain of the policy's input layer — **not** a re-validation of the environment, and **not** a re-run of the baselines.

5. Frozen vs. open — what you may and may not move

Frozen (D42), enforced by `tests/test_freeze.py`: band geometry, slot clock, dwell schedule, `N`, σ , γ , `PD_POPULATION`. **The action space follows directly from these and does not move.** Editing `rfenv/constants.py` turns the suite red, and by **D25** any change forces re-validation from gate 1 plus a re-run of every baseline.

Deliberately not frozen — the RL lane's to move: - the **reward** — three candidates already written and selectable by name: `hit_z`, `hit_y`, `first_intercept` (`rfenv/env.py:113`, verified at runtime). The selection rule is **D47**; the reward/observation asymmetry is **D29**; per-slot summation is **D31 - observation extensions** — **D30** (§4) - the per-episode scenario draw

6. Verify any of this yourself

```
.venv/bin/python -m pytest tests -q          # 229 passed, re-run 2026-09-06

.venv/bin/python -c "
from rfenv.env import ScanEnv, REWARDS
from rfenv.scenario import EmitterPool
from rfenv.baselines import OBSERVABLE_INFO
env = ScanEnv(pool=EmitterPool.from_train(), reward='hit_z')
obs, info = env.reset(seed=0)
print('action_space      ', env.action_space)          # Discrete(36)
print('observation_space', env.observation_space)      # Box((146,), float32); high is per-dim, D55
print('obs shape/range  ', obs.shape, obs.min(), obs.max())
print('rewards available', sorted(REWARDS))
print('observable info  ', sorted(OBSERVABLE_INFO))
"
```

Where the reasoning lives. The agent interface is documented in the module docstring at the top of `rfenv/env.py`. The decisions are in `docs/project/DECISIONS.md` — **D3, D12, D15, D16, D19, D20, D21, D25, D28, D29, D30, D31, D34, D35, D42, D47**. The buildable spec is `docs/project/ENVIRONMENT_SPEC.md` §L3.

Do not redefine a metric here or anywhere else. `docs/project/EVALUATION.md` is the single authority on metrics, baselines, gates and protocol — including the fact that the bar for RL is **rung 5** (interception ratio 0.1104, censored intercept time 3.20 s), not round-robin.